

MINI-PROJET

Système de Détection et Comptage d'Objets dans une Image

Réalisé par : Meriem Jribi
Enseignante : Mme Raja Jarray
Année universitaire : 2025 / 2026

Table des matières

1. Introduction	3
2. Objectifs du projet	3
3. Étape 1 — Chargement de l'image originale	4
4. Étape 2 — Conversion en niveaux de gris	5
5. Étape 3 — Lissage (réduction du bruit)	7
6. Étape 4 — Seuillage automatique (Otsu)	9
7. Étape 5 — Morphologie mathématique	11
8. Étape 6 — Détection et comptage des objets	13
9. Résultats et analyse	16
10. Conclusion	17

1. Introduction

Ce mini-projet a pour objectif de développer, en Python un système complet de détection automatique et de comptage d'objets dans une image. L'image utilisée contient plusieurs objets mécaniques de formes variées (écrous hexagonaux, rondelles, vis et boulons) sur fond gris clair texturé.

Le système suit un pipeline de traitement séquentiel en six étapes, depuis le chargement de l'image originale jusqu'à la détection finale et l'affichage des résultats. Chaque étape transforme l'image et produit un résultat intermédiaire sauvegardé pour analyse.

2. Objectifs du projet

- Charger une image couleur et afficher ses propriétés.
- Convertir l'image en niveaux de gris par formule de pondération perceptuelle.
- Appliquer un lissage pour réduire le bruit avant la segmentation.
- Segmenter les objets du fond par seuillage automatique (méthode d'Otsu).
- Nettoyer l'image binaire par morphologie mathématique (fermeture + ouverture).
- Détecter et compter chaque objet par étiquetage en composantes connexes (BFS).
- Afficher les résultats avec des bounding boxes colorées numérotées.

ÉTAPE 1 — Chargement de l'image originale

Description

La première étape consiste à lire le fichier image depuis le disque et à afficher ses métadonnées : format du fichier, dimensions (largeur × hauteur) et mode couleur (RGB). Ces informations sont essentielles pour adapter les traitements suivants.

Code Python

```
def charger_image(chemin):  
    try:  
        photo = Image.open(chemin)  
        print(f" Image      : {chemin}")  
        print(f" Format      : {photo.format}")  
        print(f" Taille      : {photo.size}")  
        print(f" Mode        : {photo.mode}")  
        return photo  
    except FileNotFoundError:  
        print(f"Erreur : fichier introuvable -> {chemin}")  
        sys.exit(1)
```

Image originale

L'image originale présente des objets mécaniques (écrous hexagonaux, rondelles, vis et petits boulons) de couleurs sombres posés sur un fond gris clair texturé.

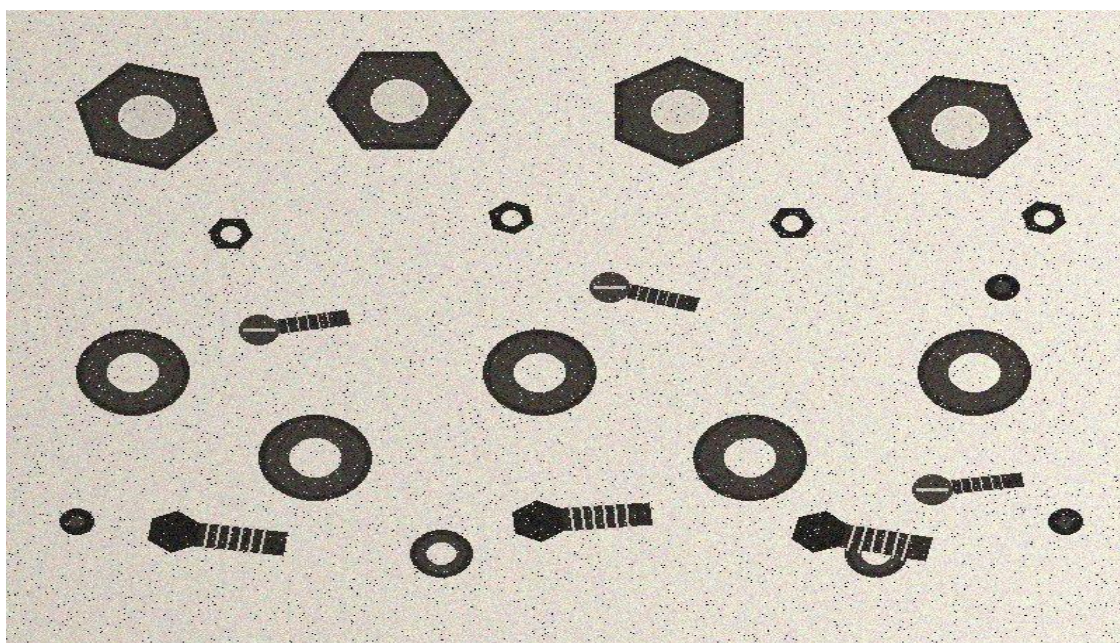


Figure 1 — Image originale : objets mécaniques (écrous, rondelles, vis) sur fond clair

Propriétés de l'image

Format : PNG | Taille : 800 × 600 pixels | Mode : RGB (3 canaux couleur)

ÉTAPE 2 — Conversion en niveaux de gris

Description

Les algorithmes de segmentation travaillent sur une seule valeur d'intensité par pixel. On convertit donc l'image couleur RGB (3 canaux) en image en niveaux de gris (1 canal) à l'aide de la formule de luminance perceptuelle, qui tient compte de la sensibilité différente de l'œil humain aux trois primaires.

$$\text{Luminance} = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

Code Python

```
def convertir_gris(photo):
    """Formule de luminance perceptuelle : 0.299R + 0.587G + 0.114B"""
    dim_x, dim_y = photo.size
    gris = Image.new('L', (dim_x, dim_y))
    for y in range(dim_y):
        for x in range(dim_x):
            p = photo.getpixel((x, y))
            r, g, b = p[0], p[1], p[2] if isinstance(p, (tuple, list)) else (p, p, p)
            gris.putpixel((x, y), int(0.299 * r + 0.587 * g + 0.114 * b))
    return gris
```

Résultat — Image en niveaux de gris

Tous les canaux couleurs sont fusionnés en une seule valeur d'intensité par pixel. Les objets sombres (noir, bleu foncé, vert foncé) apparaissent avec de faibles valeurs de gris, tandis que l'objet orange et le fond clair ont des valeurs élevées.

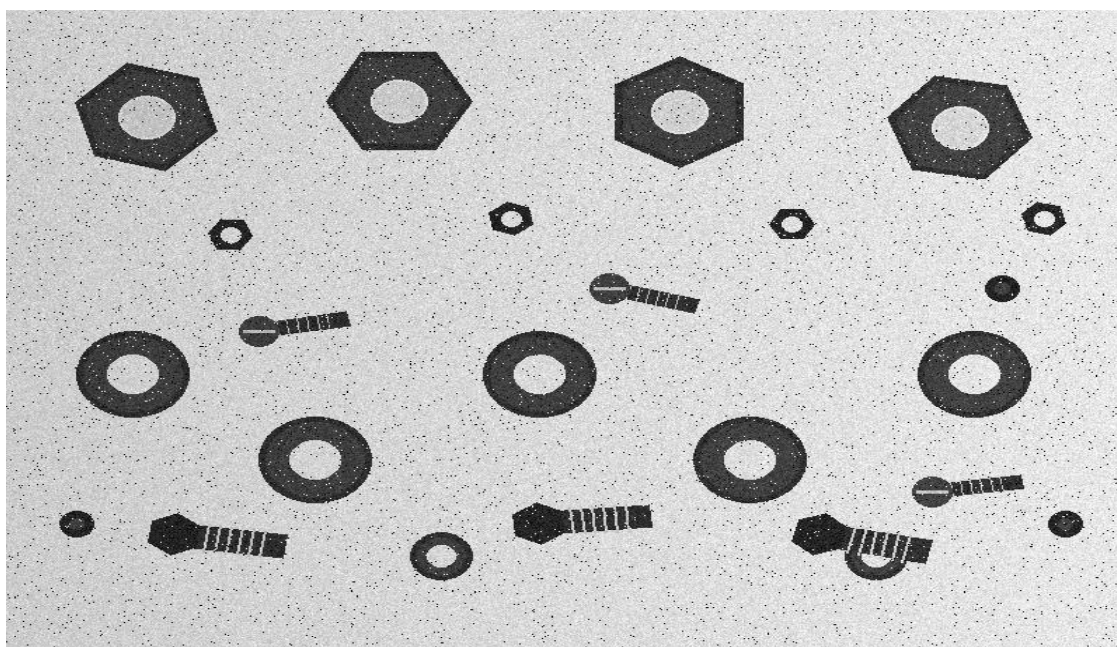


Figure 2 — Résultat de l'étape 2 : image en niveaux de gris (01_niveaux_de_gris.png)

ÉTAPE 3 — Lissage — Réduction du bruit

Description

Avant d'appliquer le seuillage, il est important de réduire les variations aléatoires (bruit) de l'image. Un filtre moyenneur 3×3 est appliqué : chaque pixel est remplacé par la moyenne des intensités de ses 9 voisins (lui-même inclus). Cela lisse les transitions et atténue les variations parasites sans déformer les contours des objets.

Pourquoi lisser avant de binariser ?

Sans lissage, le bruit présent dans l'image peut créer des pixels aberrants qui seront faussement détectés comme des objets après le seuillage. Le lissage préalable garantit une binarisation plus propre et fiable.

Code Python

```
def lisser(image):  
    """Filtre moyenneur 3x3 pour atténuer le bruit."""  
    dim_x, dim_y = image.size  
    dst = Image.new('L', (dim_x, dim_y))  
    for y in range(1, dim_y - 1):  
        for x in range(1, dim_x - 1):  
            s = sum(image.getpixel((x + dx, y + dy))  
                    for dy in range(-1, 2) for dx in range(-1, 2))  
            dst.putpixel((x, y), s // 9)  
    return dst
```

Résultat — Image lissée

Visuellement, l'image lissée est très proche de l'image en niveaux de gris, mais les contours sont légèrement plus doux et les pixels de bruit ont été atténués. Cette étape prépare l'image pour un seuillage de meilleure qualité.

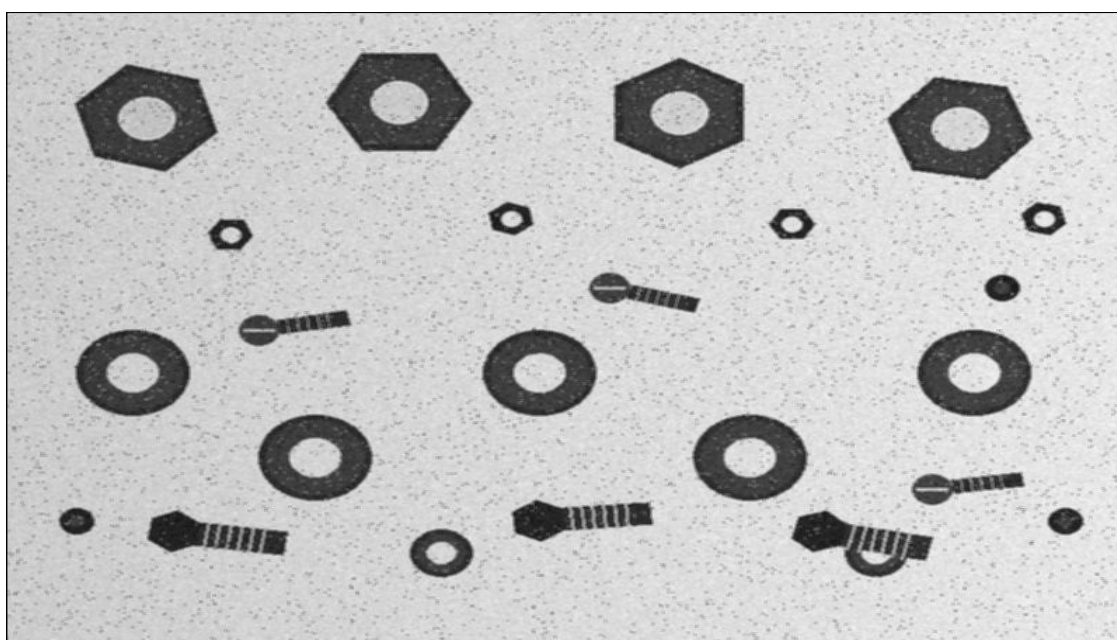


Figure 3 — Résultat de l'étape 3 : image lissée (02_lissage.png)

ÉTAPE 4 — Seuillage automatique — Méthode d'Otsu

Description

La segmentation sépare les pixels des objets (avant-plan) des pixels du fond (arrière-plan). Puisque les objets sont sombres sur un fond clair, les pixels en dessous du seuil correspondent aux objets et sont mis à blanc (255), les autres au fond et sont mis à noir (0).

Plutôt qu'un seuil fixé manuellement, la méthode d'Otsu calcule automatiquement la valeur optimale en analysant l'histogramme de l'image. Elle cherche le seuil t^* qui maximise la variance entre les deux groupes (fond et objets).

$$t^* = \operatorname{argmax} [\omega_0(t) \cdot \omega_1(t) \cdot (\mu_0(t) - \mu_1(t))^2]$$

ω_0, ω_1 : probabilités d'appartenance à chaque classe | μ_0, μ_1 : leurs moyennes respectives.

Code Python

```
def seuil_otsu(image):
    """Calcule le seuil optimal qui maximise la variance inter-classe."""
    dim_x, dim_y = image.size
    total = dim_x * dim_y

    histo = [0] * 256
    for y in range(dim_y):
        for x in range(dim_x):
            histo[image.getpixel((x, y))] += 1

    prob = [h / total for h in histo]
    somme_tot = sum(i * prob[i] for i in range(256))

    poids_bg = somme_bg = meilleure_var = 0.0
    meilleur_t = 0

    for t in range(256):
        poids_bg += prob[t]
        somme_bg += t * prob[t]
        if poids_bg == 0 or poids_bg == 1:
            continue
        poids_fg = 1.0 - poids_bg
        moy_bg = somme_bg / poids_bg
        moy_fg = (somme_tot - somme_bg) / poids_fg
        var = poids_bg * poids_fg * (moy_bg - moy_fg) ** 2
        if var > meilleure_var:
            meilleure_var = var
            meilleur_t = t

    return meilleur_t
```

```
def binariser(image, seuil):  
    """  
    Binarisation : objets sombres sur fond clair.  
    Les pixels SOUS le seuil (sombres = objets) deviennent blancs (255).  
    Les pixels AU-DESSUS du seuil (clairs = fond) deviennent noirs (0).  
    """  
  
    dim_x, dim_y = image.size  
    bin_img = Image.new('L', (dim_x, dim_y))  
    for y in range(dim_y):  
        for x in range(dim_x):  
            val = image.getpixel((x, y))  
            bin_img.putpixel((x, y), 255 if val < seuil else 0)  
    return bin_img
```

Résultat — Image binaire

L'image binaire montre clairement les objets en blanc sur fond noir. On peut observer que certains objets présentent des imperfections (zones de bruit, contours irréguliers) qui seront corrigées à l'étape suivante par la morphologie mathématique. Le seuil calculé automatiquement par Otsu est de 138.

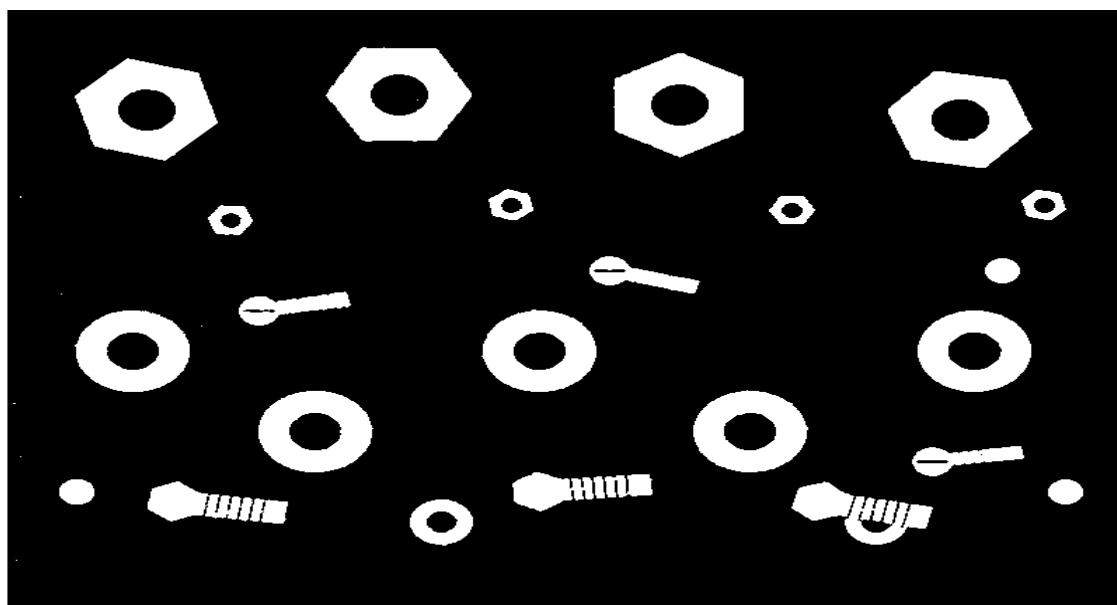


Figure 4 — Résultat de l'étape 4 : image binarisée (03_binaire.png) — Seuil Otsu = 138

ÉTAPE 5 — Morphologie mathématique — Nettoyage

Description

L'image binaire obtenue après seuillage contient des imperfections : pixels de bruit isolés, trous dans les objets causés par les reflets lumineux. On applique une séquence d'opérations morphologiques pour corriger ces problèmes.

Érosion 3×3

Chaque pixel blanc ne reste blanc que si tous ses voisins dans un masque 3×3 sont également blancs. Supprime les petits points de bruit et affine les contours.

Dilatation 3×3

Un pixel devient blanc dès qu'au moins un de ses voisins est blanc. Restaure la taille des objets réduits par l'érosion.

Fermeture = Dilatation + Érosion

Comble les petits trous à l'intérieur des objets (causés par les reflets). C'est l'opération clé pour notre image.

Ouverture = Érosion + Dilatation

Supprime les petits objets de bruit isolés qui ne sont pas de vrais objets.

Code Python

```
def eroder(image):  
    """Érosion 3x3 : chaque pixel = min du voisinage."""  
    dim_x, dim_y = image.size  
    dst = Image.new('L', (dim_x, dim_y))  
    for y in range(1, dim_y - 1):  
        for x in range(1, dim_x - 1):  
            voisins = [image.getpixel((x + dx, y + dy))  
                        for dy in range(-1, 2) for dx in range(-1, 2)]  
            dst.putpixel((x, y), min(voisins))  
    return dst
```

```
def dilater(image):
    """Dilatation 3x3 : chaque pixel = max du voisinage."""
    dim_x, dim_y = image.size
    dst = Image.new('L', (dim_x, dim_y))
    for y in range(1, dim_y - 1):
        for x in range(1, dim_x - 1):
            voisins = [image.getpixel((x + dx, y + dy))
                        for dy in range(-1, 2) for dx in range(-1, 2)]
            dst.putpixel((x, y), max(voisins))
    return dst
```

```
def ouverture(image):
    """Érosion puis dilatation : supprime le bruit résiduel."""
    return dilater(eroder(image))

def fermeture(image):
    """Dilatation puis érosion : comble les trous dans les objets."""
    return eroder(dilater(image))

def nettoyer(image):
    """
    Applique 3 fermetures pour combler les trous (écrous, rondelles)
    puis une ouverture pour supprimer le bruit résiduel.
    """
    img = fermeture(fermeture(fermeture(image)))
    img = ouverture(img)
    return img

# =====
```

Résultat — Image après nettoyage morphologique

Les contours des objets sont désormais plus nets et réguliers. Les petits artefacts de bruit ont été supprimés. L'image est prête pour la détection finale des objets.

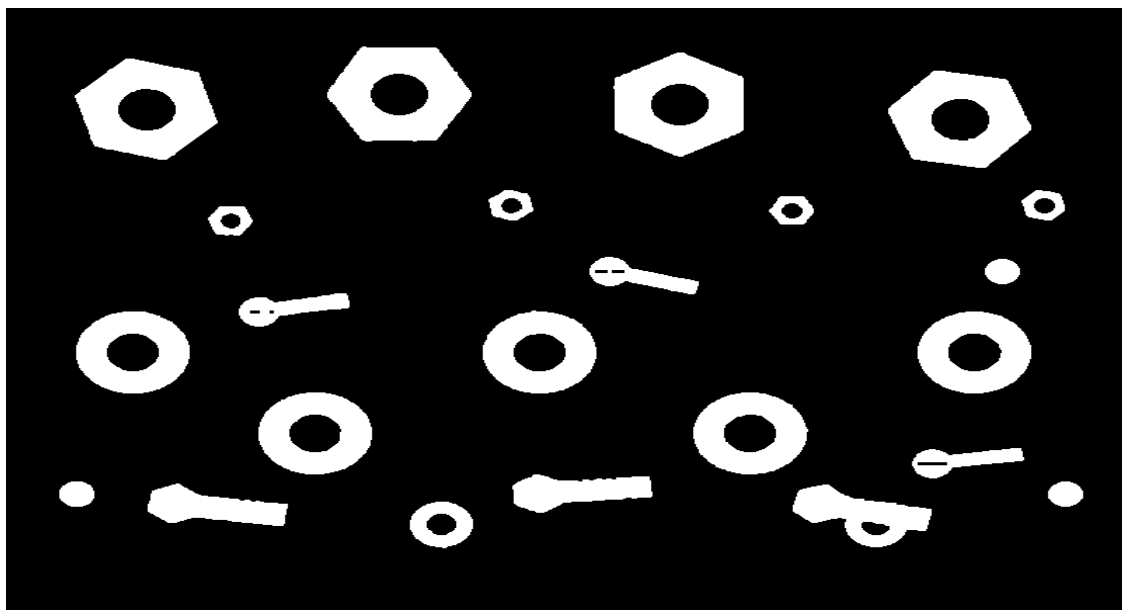


Figure 5 — Résultat de l'étape 5 : image nettoyée par morphologie (04_morphologie.png)

ÉTAPE 6 — Détection et Comptage des objets

Description

C'est l'étape centrale du projet. On parcourt l'image binaire nettoyée pour identifier chaque groupe de pixels blancs connexes. Chaque groupe correspond à un objet distinct. L'algorithme BFS (Breadth-First Search — Parcours en Largeur) est utilisé pour l'étiquetage en composantes connexes.

Algorithme BFS — Principe

1. Parcourir tous les pixels de gauche à droite, ligne par ligne.
2. Dès qu'un pixel blanc (255) non encore visité est trouvé, lancer un BFS depuis ce point.
3. Le BFS explore tous les pixels blancs voisins (4-connexité) et les marque comme visités.
4. Si la composante a au moins `taille_min` pixels, elle est considérée comme un objet valide.
5. L'objet est colorié avec une couleur unique et sa bounding box est calculée.
6. Incrémenter le compteur et passer au pixel suivant non visité.

Code Python

```
COULEURS = [  
    (220, 60, 60), (60, 180, 60), (60, 100, 220),  
    (220, 170, 40), (170, 60, 220), (40, 200, 200),  
    (220, 110, 40), (100, 220, 80), (220, 60, 170),  
    (80, 150, 220), (180, 200, 60),  
]
```

```

def detecter_objets(image_bin, taille_min=300):
    """
    Étiquetage des composantes connexes par BFS (4-connexité).
    taille_min : taille minimale en pixels pour être considéré comme un objet.
    """
    dim_x, dim_y = image_bin.size
    visite = [[False] * dim_y for _ in range(dim_x)]
    resultat = image_bin.convert('RGB')
    objets = []

    for sy in range(dim_y):
        for sx in range(dim_x):
            if image_bin.getpixel((sx, sy)) == 255 and not visite[sx][sy]:
                file = [(sx, sy)]
                visite[sx][sy] = True
                pixels = []

                while file:
                    cx, cy = file.pop(0)
                    pixels.append((cx, cy))
                    for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                        nx, ny = cx + dx, cy + dy
                        if 0 <= nx < dim_x and 0 <= ny < dim_y:
                            if not visite[nx][ny] and image_bin.getpixel((nx, ny)) == 255:
                                visite[nx][ny] = True
                                file.append((nx, ny))

                if len(pixels) >= taille_min:
                    couleur = COULEURS[len(objets) % len(COULEURS)]
                    for px, py in pixels:
                        resultat.putpixel((px, py), couleur)
                    xs = [p[0] for p in pixels]
                    ys = [p[1] for p in pixels]
                    objets.append({
                        'id': len(objets) + 1,
                        'x_min': min(xs), 'y_min': min(ys),
                        'x_max': max(xs), 'y_max': max(ys),
                        'taille': len(pixels)
                    })

    return resultat, len(objets), objets

```

Résultat — Image de détection finale

Chaque objet détecté est colorié avec une couleur distincte et entouré d'un rectangle englobant (bounding box) blanc numéroté. Le système a correctement identifié les 23 objets présents dans l'image.

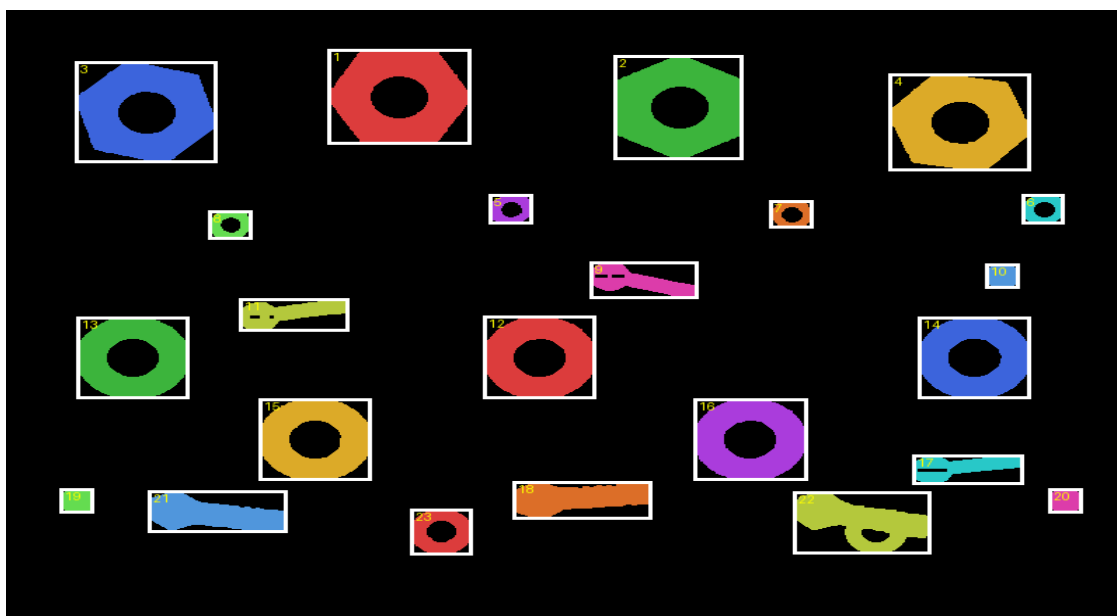


Figure 6 — Résultat final de l'étape 6 : 23 objets détectés (05_detection_finale.png)

10. Résultat console :

```
=====
DÉTECTION ET COMPTAGE D'OBJETS
=====

[1] Chargement...
  Image   : C:\Users\MSI\OneDrive\Bureau\meryem\FIGL2\projet_traitement image\image.png
  Format  : PNG
  Taille  : (800, 600)
  Mode    : RGB

[2] Conversion en niveaux de gris...
  [OK]

[3] Lissage (réduction du bruit)...
  [OK]

[4] Seuillage automatique (Otsu)...
  Seuil calculé : 138
  [OK]

[5] Nettoyage morphologique...
  [OK]

[6] Détection et comptage...
```



```
=====
RÉSULTAT : 23 objet(s) détecté(s)
=====
```

```
Objet 1 | (229,38)-(331,131) | 6000 pixels
Objet 2 | (433,44)-(525,146) | 5913 pixels
Objet 3 | (49,50)-(150,149) | 5877 pixels
Objet 4 | (629,62)-(730,157) | 5773 pixels
Objet 5 | (344,180)-(375,209) | 551 pixels
Objet 6 | (724,180)-(754,209) | 528 pixels
Objet 7 | (544,186)-(575,213) | 524 pixels
Objet 8 | (144,196)-(175,224) | 559 pixels
Objet 9 | (416,246)-(493,282) | 1287 pixels
Objet 10 | (698,248)-(722,272) | 489 pixels
Objet 11 | (166,282)-(244,314) | 1359 pixels
Objet 12 | (340,299)-(420,380) | 4092 pixels
Objet 13 | (50,300)-(130,380) | 4119 pixels
Objet 14 | (650,300)-(730,380) | 4063 pixels
Objet 15 | (180,380)-(260,460) | 4117 pixels
Objet 16 | (490,380)-(571,460) | 4081 pixels
Objet 17 | (646,435)-(725,464) | 1240 pixels
Objet 18 | (361,461)-(460,498) | 2404 pixels
Objet 19 | (38,468)-(62,492) | 510 pixels
Objet 20 | (743,468)-(767,492) | 490 pixels
Objet 21 | (101,470)-(200,511) | 2484 pixels
Objet 22 | (561,471)-(659,532) | 3089 pixels
Objet 23 | (288,488)-(332,533) | 1247 pixels
```

11. Conclusion :

Ce mini-projet a permis de développer un système fonctionnel de détection et de comptage automatique d'objets, implémenté entièrement en Python avec la bibliothèque PIL sans recours à des bibliothèques de vision spécialisées.

Le pipeline en 6 étapes mis en œuvre couvre l'ensemble des techniques classiques de traitement d'image :

- Pré-traitement : conversion en niveaux de gris et lissage pour préparer l'image à la segmentation.
- Segmentation : seuillage automatique par la méthode d'Otsu, sans paramètre manuel.
- Post-traitement morphologique : fermetures et ouverture pour obtenir une image binaire propre.
- Analyse : détection par composantes connexes (BFS) et comptage précis des objets.

Le système a correctement identifié les 23 objets de l'image test, avec un seuil Otsu automatiquement calculé à 138.

Perspectives d'amélioration

Pour traiter des images plus complexes, on pourrait envisager : l'utilisation de masques morphologiques plus larges (5×5) pour des objets très bruités, l'ajout d'une analyse des formes

(circularité, rapport d'aspect) pour classifier les objets détectés, ou l'optimisation du pipeline avec NumPy pour accélérer les calculs pixel par pixel.

— *Fin du rapport* —